CyberVault X

Secure Password Manager with End-to-End Encryption

Version: 5.6.0 Professional Release
Course: CET334 - Cryptographic Algorithms & Protocols
Project: Project 4 - Secure Password Manager
Delivery: working desktop application, code, tests, documentation, presentation, and live demo.

Core Requirement	Implementation
Encrypted Password Vault	AES-GCM field encryption before SQLite storage.
Master Password Protection	PBKDF2-SHA256 with password policy and unlock throttling.
Password Generation	Configurable generator with entropy feedback.
Breach Detection	Offline SHA1 breach hash list for local checks.
Strength Analysis	Weak, reused, breached, stale, and metadata risk scoring.

1. Executive Summary

CyberVault X is a local-first password manager designed to protect user credentials using end-to-end encryption, master-password protection, password generation, breach checking, and security posture analysis. It satisfies the original academic requirements and adds a professional product layer: AI Guardian, Security Proof Center, encrypted backups, tamper-evident audit logs, privacy-safe reports, and report-package verification.

2. Problem Statement

Users often reuse weak passwords across many services. When one service is breached, attackers can use credential stuffing to compromise other accounts. CyberVault X reduces this risk by storing credentials safely, generating strong passwords, warning about weak/reused/breached passwords, and guiding the user through remediation.

3. Project Objectives

The project objectives are: build an encrypted vault, protect it with a master password, use AES-GCM for authenticated encryption, provide a strong password generator, perform offline breach detection, generate safe reports, and provide proof screens that demonstrate the security controls during the live demo.

4. Tools and Technologies

Area	Technology
Language	Python 3.11+
Desktop GUI	Tkinter, chosen as a lightweight built-in alternative to PyQt5
Encryption	PyCryptodome AES-GCM
Key Derivation	PBKDF2-SHA256
Storage	SQLite
Testing	pytest / unittest
Packaging	PyInstaller

GUI note: the original idea suggested PyQt5. This implementation uses Tkinter to keep the project portable while preserving the same functional and security objectives.

5. Architecture

CyberVault X uses a layered architecture: UI layer, manager/orchestration layer, service layer, cryptographic layer, database layer, and automated testing layer. This separation keeps the cryptographic logic independent from the interface and makes future UI migration possible.

Layer	Responsibility
UI	Desktop screens, dialogs, workflow controls, presentation mode.
Manager	Vault actions, authentication, settings, and service coordination.
Services	Backup, reporting, AI Guardian, proof checks, and product intelligence.
Crypto	AES-GCM, backup encryption, master verification, passphrase policy.
Database	SQLite schema, migrations, metadata, credentials, history, audit logs.

6. Cryptographic Design

Master Password

The master password is never stored directly. PBKDF2-SHA256 derives cryptographic material from the password and a random salt.

AES-GCM Field Encryption

Sensitive credential fields are encrypted before database storage. Each field is stored as a nonce/cipher pair and authenticated during decryption.

Additional Authenticated Data

AAD binds encrypted data to the credential id and field name, reducing ciphertext-swapping risks between fields or records.

Encrypted Backup

Backups use a dedicated encrypted envelope format. Restore preview and rollback protect the active vault from partial import failures.

7. Main Features

Feature	Value
Encrypted Vault	Add, edit, delete, restore, and search credentials while keeping sensitive fields encrypted at rest.
Security Center	Detect weak, reused, breached, stale, and metadata-incomplete credentials.
AI Guardian	Local deterministic advisor that turns findings into prioritized actions and attacker-view explanatio
Security Proof Center	Verifies encrypted schema, AAD posture, audit chain, report package, and backup preview.
Tamper-Evident Audit	Activity events are linked with hashes to detect modification, deletion, or reordering.
Privacy-Safe Reports	Exports executive evidence without raw passwords and with manifest hashes.

8. Database Design

The SQLite database contains app metadata, encrypted credentials, encrypted credential history, tamper-evident activity logs, and schema migration tracking. The schema avoids plaintext password columns and supports future migrations.

9. Testing and Validation

Automated tests cover encryption, AAD protections, backup roundtrip, backup rollback, report manifest verification, AI Guardian redaction, security scoring, and audit-chain behavior.

Recommended commands:

```
python -m pytest -q tests
python tools/release_preflight.py
build_release.bat
```

10. Live Demo Scenario

Launch the application, create/unlock a vault, load the Professional Demo Vault, explain the Dashboard score, review Security Center findings, generate an AI Guardian plan, replace a weak password, open Security Proof Center, export and verify a report package, export an encrypted backup, preview restore impact, and demonstrate panic lock.

11. Limitations

Limitation	Professional Handling
Breach dataset is demo-sized	Documented honestly; future work supports larger offline import.
Python memory handling	Auto-lock and clipboard clearing reduce exposure; full zeroization is limited in Python.
HMAC report signing	Strong local integrity proof; future work can add Ed25519 public verification.
Tkinter instead of PyQt5	Chosen for portability; service layers can be reused in a PyQt5 interface.

12. Future Improvements

Argon2id migration path, custom offline breach-list import wizard, Ed25519 public/private report signatures, secure-memory improvements, deeper UI component refactor, Windows screenshot automation, and optional PyQt5 interface variant.

13. Conclusion

CyberVault X satisfies the core secure password manager requirements and extends them with professional security workflows. The project demonstrates applied cryptography, local privacy protection, secure software design, test coverage, report generation, and live-demo readiness.